

Predictive Cloud Resource Optimizer

Predicting how busy a server will be 5 minutes from now, so extra servers can be started **before** a traffic spike — not after it.

THIS REPORT COVERS

Step 1.1 Setup
Step 1.2 Data
Step 1.3 Data preparation

DATASET

Bitbrains fastStorage
1,250 real servers, every 5 minutes

STATUS

Data is ready and checked.
Next: build the LSTM model.

CHOSEN SERVER

VM 270
8,640 clean readings (30.0 days)

1 Summary

What the project is

Cloud servers get sudden spikes of users. Normal auto-scaling waits until the server is already overloaded, then starts a new server — which takes 60–90 seconds to be ready. Users feel the slowdown during that time.

Our system uses an **LSTM** (a neural network for time-based data) to predict the load **5 minutes ahead**. If a spike is coming, servers are started early. At the end we will compare **proactive** (prediction-based) and **reactive** (normal) scaling on: how fast capacity is ready, how much capacity is wasted, and how many false alarms happen.

Progress

STEP	WHAT	STATUS	NOTES
1.1	Setup	MOSTLY DONE	README and git still missing
1.2	Data: choose VM, clean it	MOSTLY DONE	Synthetic data generator still missing
1.3	Prepare data for model	DONE	All checks pass
1.4	LSTM model	NEXT	
1.5–1.8	Evaluation, scaling, dashboard, final report	LATER	

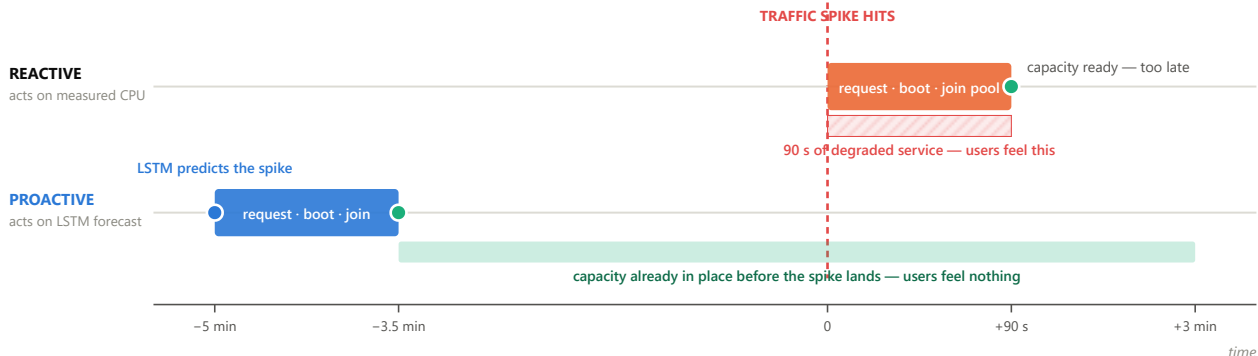
Key numbers

WHAT	NUMBER	MEANING
VM files checked	1,250	All of them, not a sample
VMs that are almost idle	889	76% of the dataset — useless for training
VMs in our target range	158	Average CPU between 20% and 60%
Chosen VM	270	Top of the list after every check
Clean readings	8,640	Exactly 30.0 days, 5 minutes apart, no gaps
Training examples (windows)	5,988	Plus 1,236 for validation and 1,236 for testing
Reconstruction error	0.00024	Scaling works correctly (only rounding)

2 How the system fits together

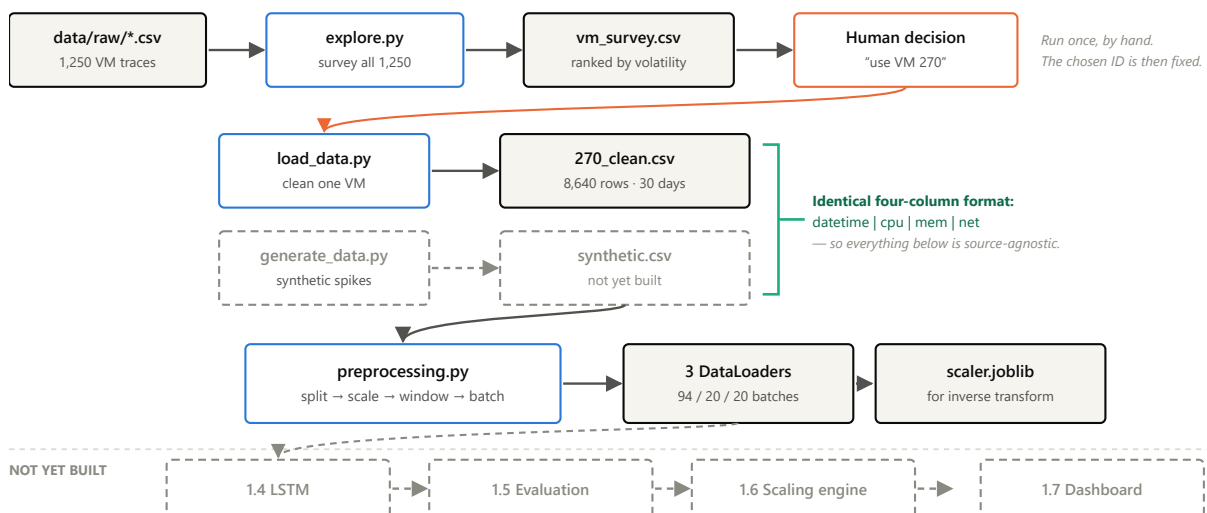
Why a 60–90 second provisioning delay decides the whole argument

The spike arrives at $t = 0$. A reactive scaler cannot start provisioning until then; a forecast starts the clock five minutes earlier.



Solid boxes are built and working. Dashed boxes are planned. The real data and the synthetic data both produce the same 4-column table, so all later steps work with either one.

End-to-end data flow — solid outline is built and verified, dashed is scheduled work



Two phases

Phase 1 (this semester) is a complete normal system. **Phase 2** (next semester) will add a small 4-qubit quantum circuit at one fixed point in the model and compare it with Phase 1. So Phase 1 must stay clean and unchanged. (The exact insertion point is shown in the study guide.)

3 Step 1.1 — Setup

Goal: a project anyone can download and run without asking questions.

ITEM	STATUS	DETAIL
Python virtual environment	DONE	venv/
Folders	DONE	data/raw , data/processed , models , plots , utils
Libraries installed	DONE	numpy, pandas, matplotlib, scikit-learn, PyTorch (CPU), streamlit, plotly, joblib
requirements.txt	DONE	List of exact library versions
.gitignore	DONE	Keeps the 128 MB dataset and model files out of GitHub
README.md	MISSING	Setup steps, data download link, data credit
Git repository	MISSING	No commits yet
notebooks/ , dashboard/	MISSING	Not needed until later

Folder structure

MiniProject/		
├─ data/raw/	1,250 original VM files (never changed, not uploaded)	
├─ data/processed/	vm_survey.csv, 270_clean.csv	
├─ models/	scaler.joblib (later: trained model)	
├─ plots/	all saved graphs	
├─ report/	source of these PDFs	
├─ utils/		
│ └─ explore.py	measure all VMs, pick one, draw graphs	BUILT
│ └─ load_data.py	clean one VM into 4 columns	BUILT
│ └─ generate_data.py	synthetic data	MISSING
│ └─ preprocessing.py	split, scale, window, batch, check	BUILT
├─ requirements.txt		
└─ .gitignore		

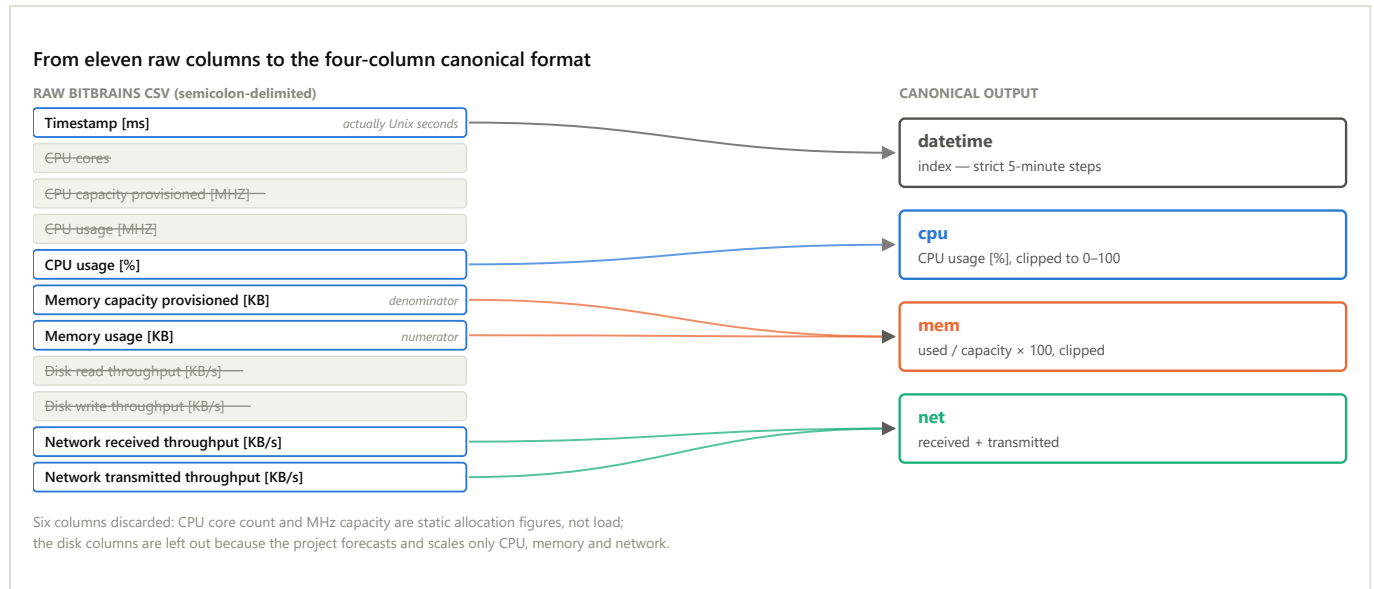
SETUP CHOICES

- **Runs on a laptop, offline, CPU only** — no cloud or GPU needed, so anyone can repeat it.
- **Data is not uploaded to GitHub** — it is too big (about 128 MB). The README will give the download link.
- **Phase 2 libraries (PennyLane) not installed yet** — keeps Phase 1 clean.

4 Step 1.2 — Getting and cleaning the data

4.1 The dataset

Bitbrains was a Dutch company that hosted servers for banks and insurance companies. TU Delft published real readings from 1,250 of these servers (the **fastStorage** set), one reading every 5 minutes. Each server is one CSV file with 11 columns. We keep what we need and make 4 columns:



Why CPU, memory and network together? When a job starts, all three usually rise together. Seeing all three gives the model more clues than CPU alone. **Why memory as a percentage?** Raw memory is in millions of KB while CPU is 0–100; without converting, memory would look more important just because its numbers are bigger.

4.2 Choosing a VM (`utils/explore.py`)

Most servers are almost idle. A model trained on an idle server just learns to say "about 2%" and learns nothing about spikes. So we checked all 1,250 files.

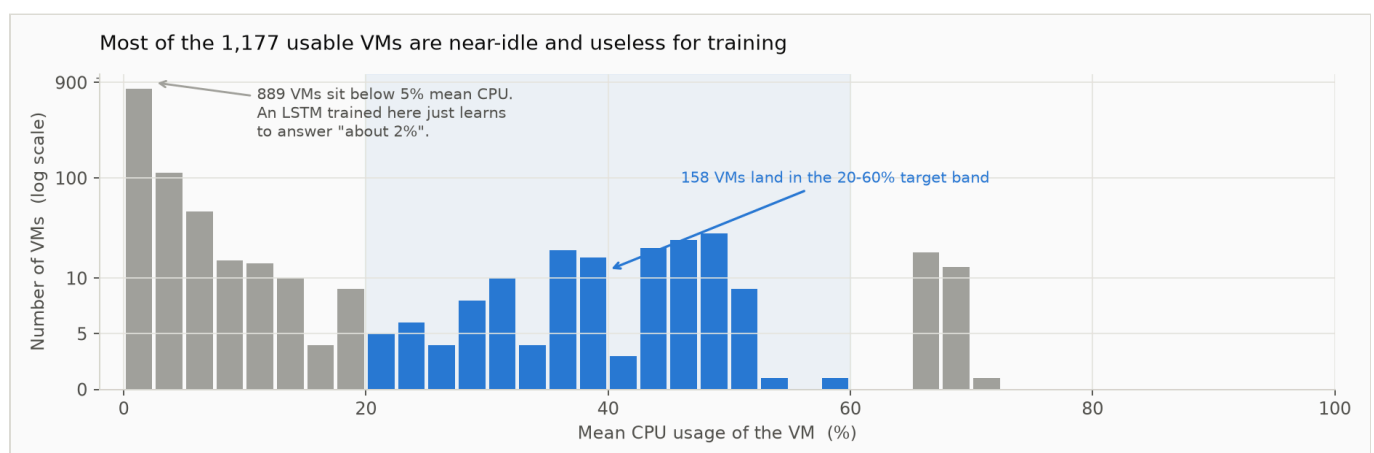


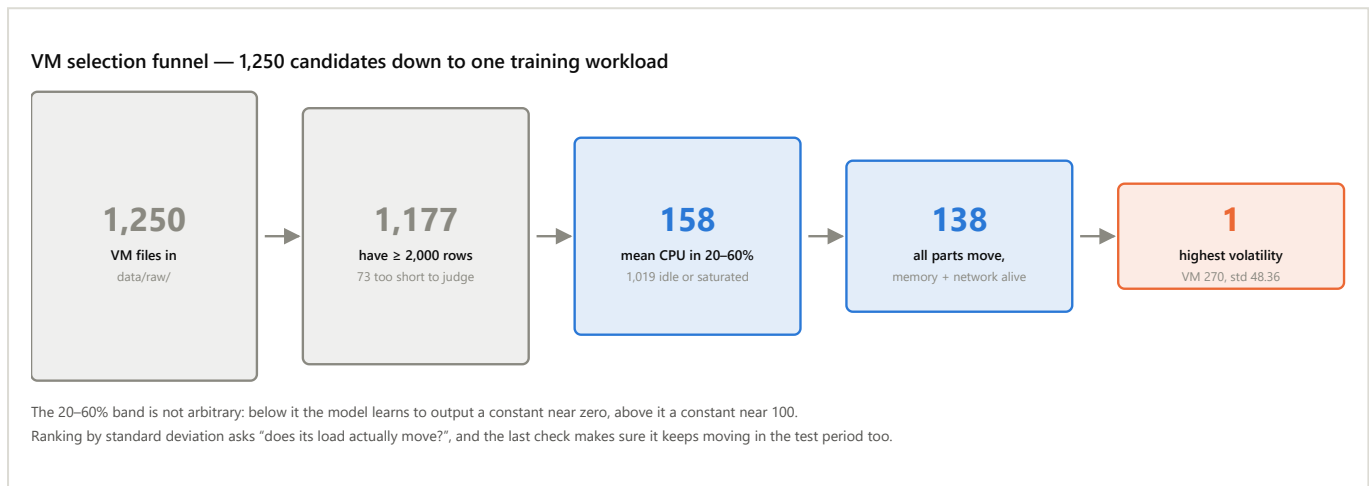
Figure 1. Average CPU of all VMs. 889 VMs (76%) are below 5%. Blue = our target range.

Three rules:

- 1 At least **2,000 readings** (about 7 days) → 1,177 VMs.

- 2 Average CPU between **20% and 60%** (not idle, not always full) → 158 VMs.
- 3 Sort by **standard deviation**, highest first (the CPU that moves most).
- 4 Check that **all three parts move** — training, validation and test. A VM that goes idle for its last days would be tested on a flat line, which proves nothing.
- 5 **Memory and network must not be dead.** The model predicts all three metrics, so a metric stuck on one value would be a column with nothing to learn. A metric is rejected if it never changes, or if one value covers more than 90% of its readings. → caught **12** VMs with dead memory and **225** with dead network across the dataset.

→ 20 rejected by these last checks, leaving **138** real candidates, led by **VM 270**.



Fix made during the work: the plan said to check only the first 100–200 files. But file names sort as text (1, 10, 100, 1000, 11...), so that is not a fair sample — it found zero good VMs. Checking all files found 158 and took about a minute.

4.3 Best vs average vs worst VM

Standard deviation (std) measures how much values spread out. The bars below are the **real readings** of three VMs on the same scale — for each CPU value, how often it actually happened. The mean (orange) and the mean ± 1 std band (blue) are drawn on top. A wide band over spread-out bars means the CPU moves a lot; a thin band over one tall bar means it hardly moves.

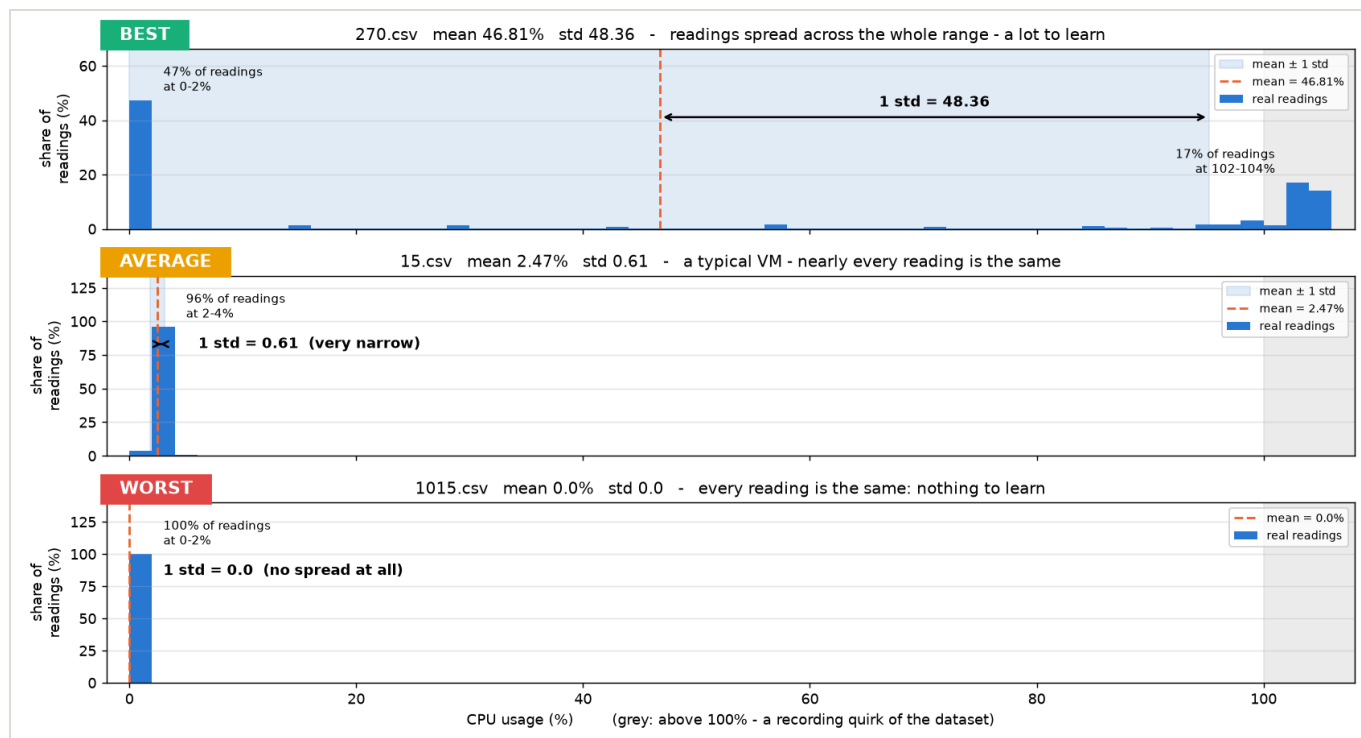


Figure 2. Best (VM 270, std 48.36): readings at both ends, so a very wide band. **Average** (VM 419, std 0.63): 99% of readings in one bar. **Worst** (VM 671, std 0.0): 100% in one bar. No bell curve is drawn because the data is not bell-shaped — 92% of VM 270's readings sit at the two extremes (53% at 0–2%, 39% above 98%).

4.4 The top candidates over one week

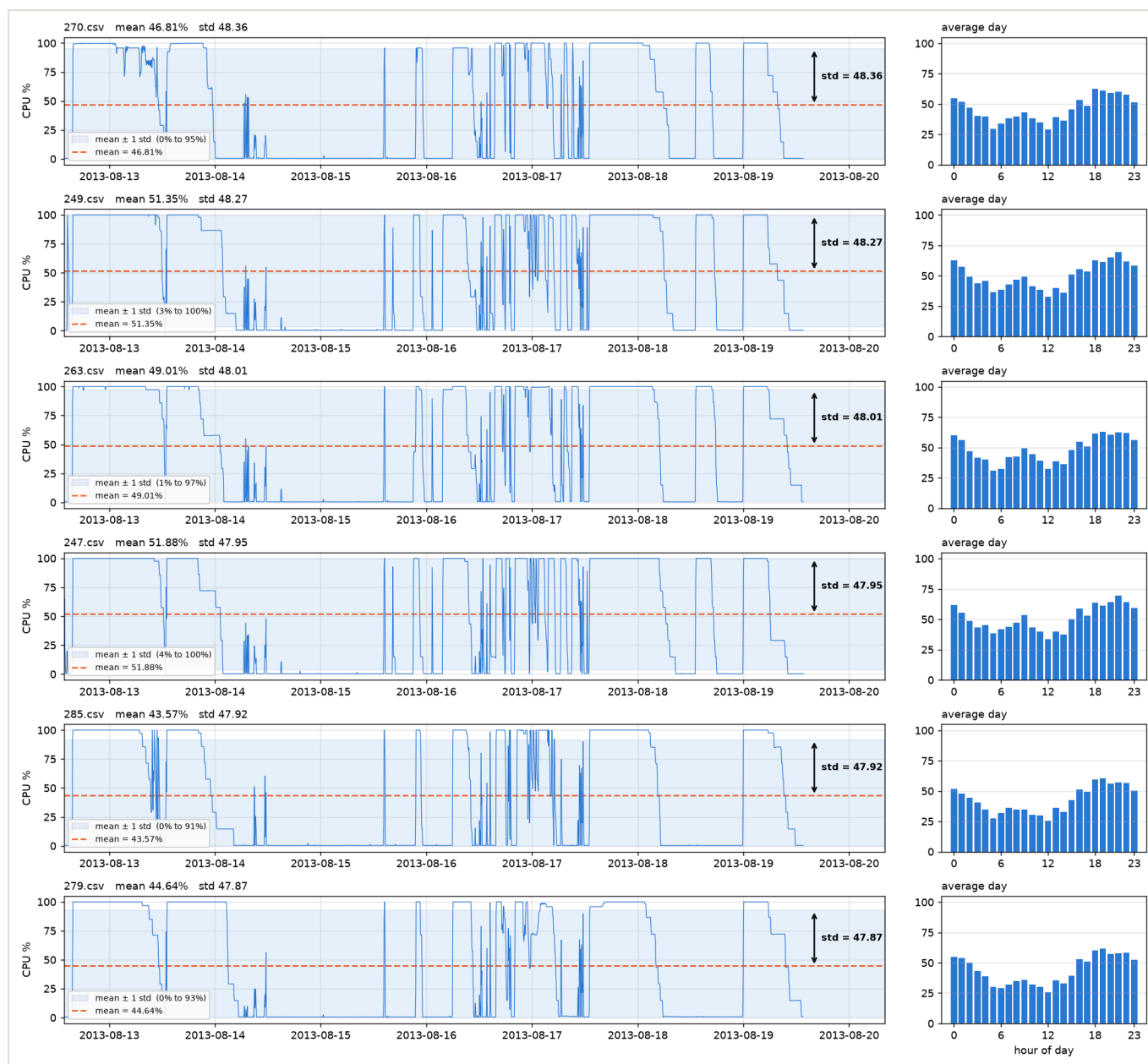
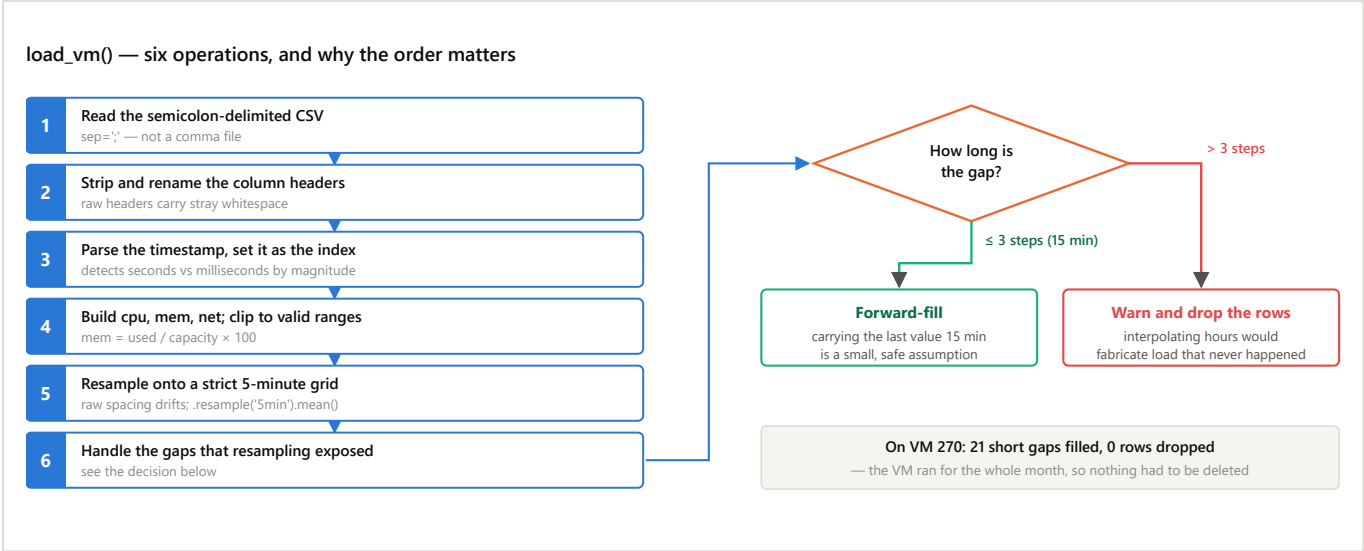


Figure 3. Top 6 candidates. Left: one week of CPU with mean (orange), ± 1 std band (blue) and std arrow. Right: average CPU for each hour of the day. These VMs **do** show a daily rhythm — quiet in the morning, busiest in the evening (VM 270 averages 29% at 11:00 and 62% at 20:00). They still look very similar to each other, so choosing between them matters little.

4.5 Cleaning VM 270 (utils/load_data.py)



RESULT	VALUE	MEANING
Clean readings kept	8,640	30.0 days (12 Aug – 11 Sep 2013)
Short gaps filled	21	Gaps of 15 minutes or less, filled with the last value
Rows removed	0	VM 270 was never switched off
Time gaps remaining	0	Every row is exactly 5 minutes after the one before

TWO DETAILS THAT WERE HANDLED

- The time column says "milliseconds", but the numbers are really **seconds**. The code detects this; otherwise all dates would be in 1970.
- Long gaps would be **deleted, not guessed** — guessing hours of data would create fake load patterns. VM 270 needed none of that: nothing had to be deleted.

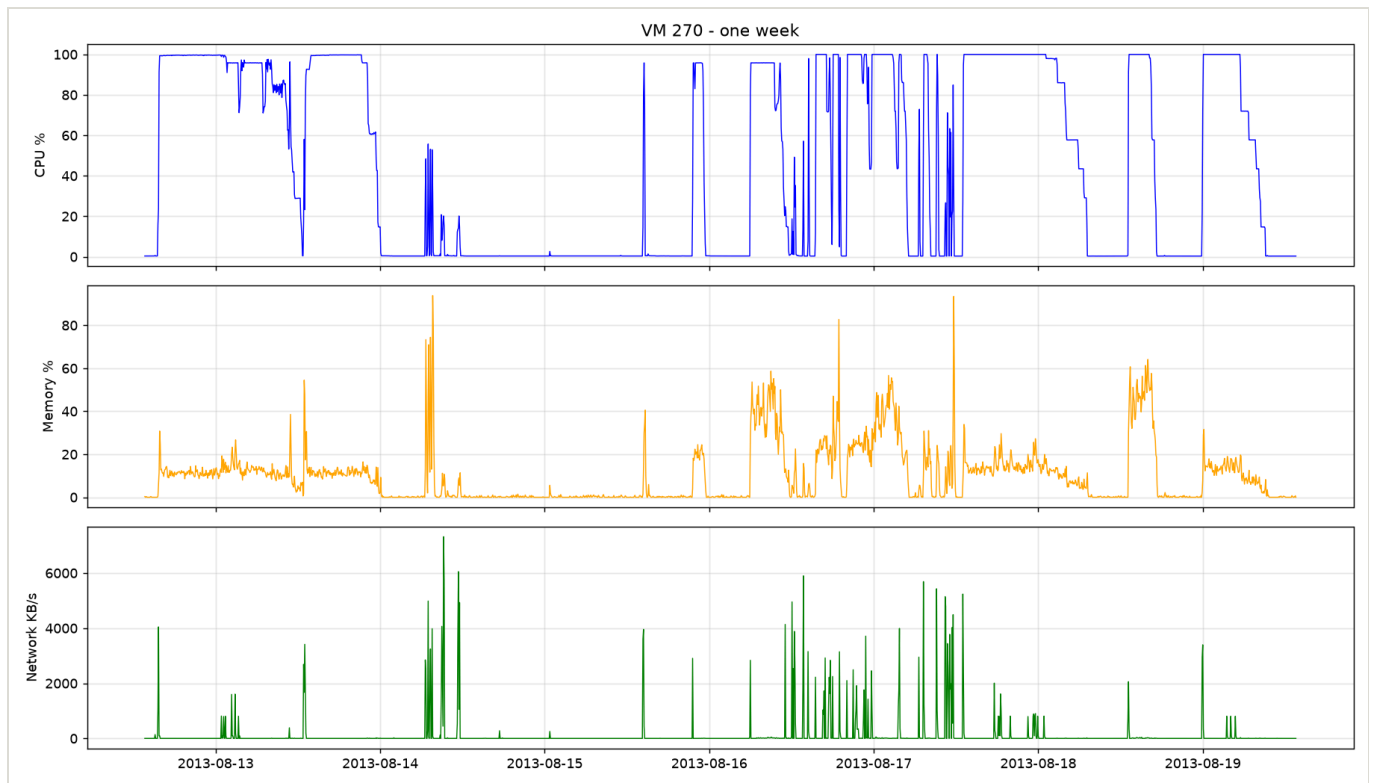


Figure 4. One week of clean VM 270 data: CPU, memory and network.

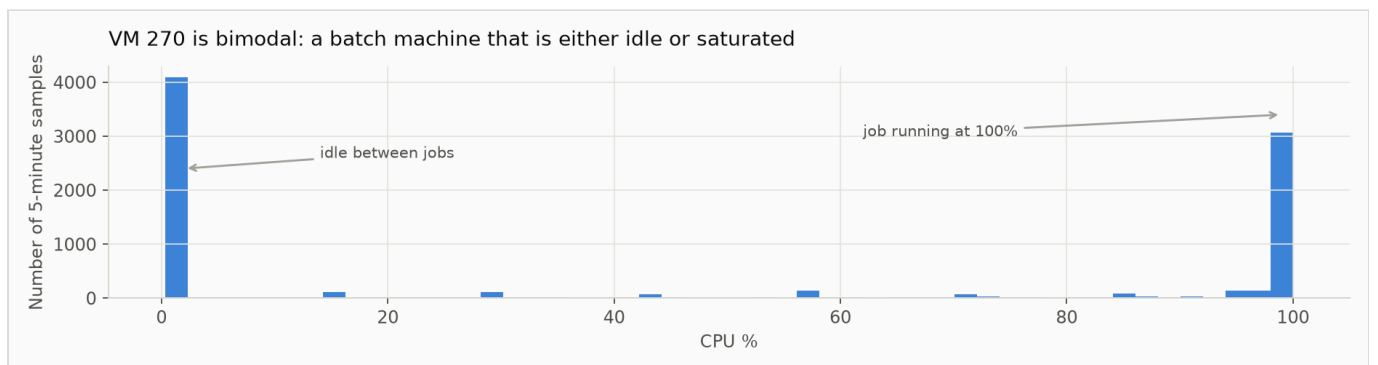


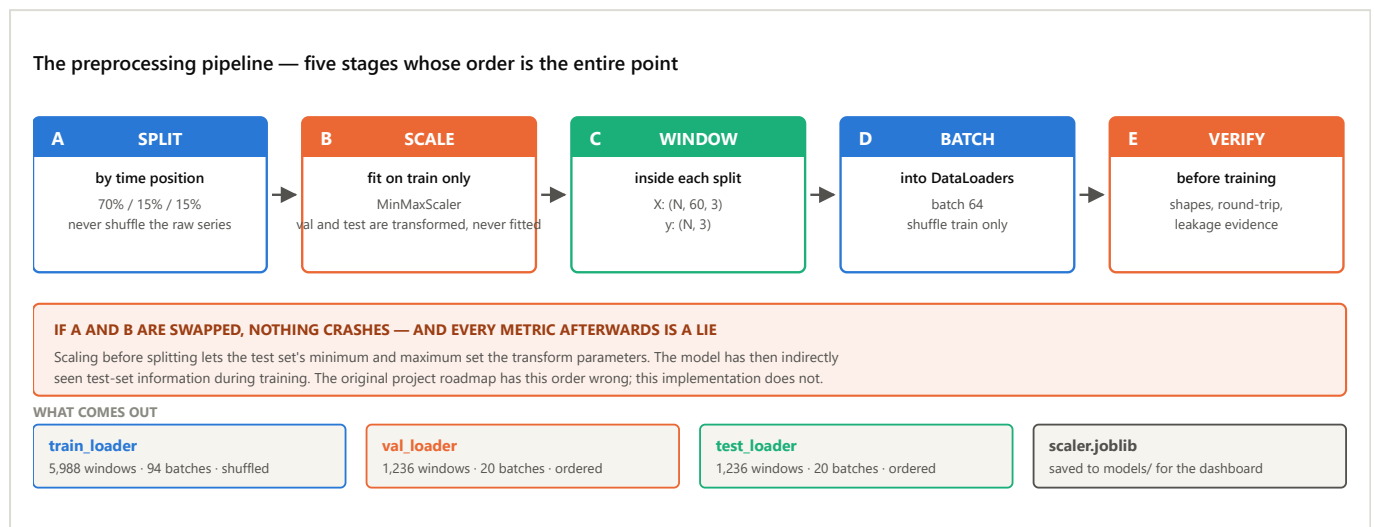
Figure 5. VM 270 is a batch machine: its CPU is almost always either 0% (idle) or 100% (running a job).

4.6 Still to do in this step

Synthetic data generator (`utils/generate_data.py`). It will create made-up data with spikes at known times, in the same 4-column format. It is needed for Step 1.6, where we must measure exactly how early the proactive system reacts to a spike.

5 Step 1.3 — Preparing the data for the model

File: `utils/preprocessing.py`. It turns the clean table into batches the LSTM can train on. **The order of the steps is very important.**



A. Split by time (70% / 15% / 15%)

First 70% of the time for training, next 15% for validation, last 15% for testing. Never random: the model must learn from the past and be tested on the future, like in real life.

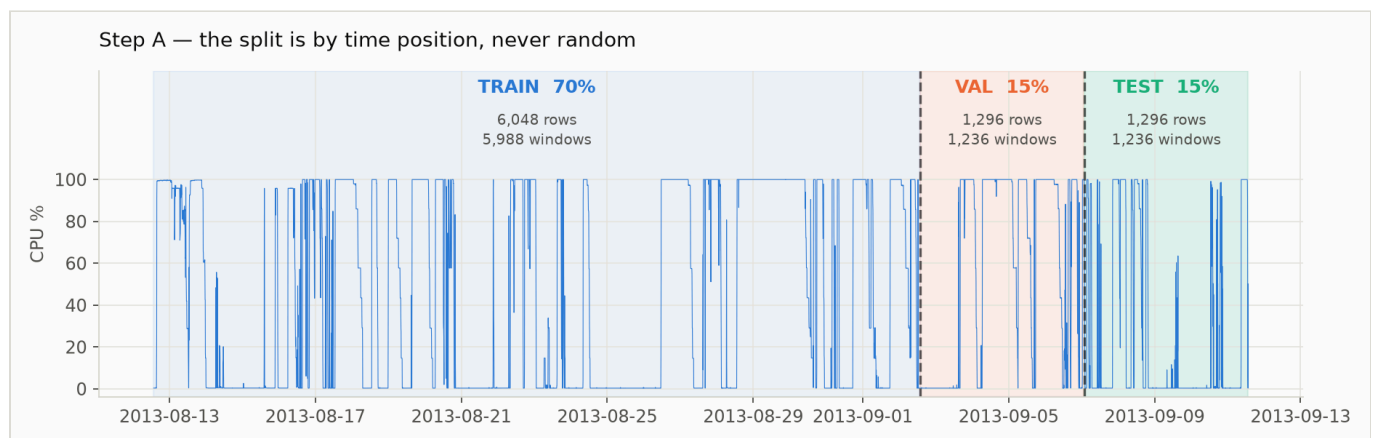


Figure 6. The time split on VM 270's CPU data.

B. Scale to 0–1 using training data only

The scaler learns the min and max from the **training** part only, then applies them to all three parts. If it also looked at the test part, test information would leak into training and the results would look better than they really are.

Why the scaler is fitted on the training split alone

✗ WRONG — scale, then split

`scaler.fit_transform(whole_series)`

one scaler fitted over everything

The maximum inside the test set helped decide the transform. The model has now indirectly seen the test data before it was ever evaluated on it.

Symptom: test data also scales to a tidy 0.00–1.00.
The tidiness is the tell.

✓ RIGHT — split, then scale

`fit_transform(train) · transform(val) · transform(test)`

scaler fitted here only

unseen

The scaler's parameters come from the past only. Test values are free to fall outside the 0–1 range, and on this dataset they do.

Measured: train 0.00–1.00, test 0.00–1.91.
The overshoot is the proof of no leakage.

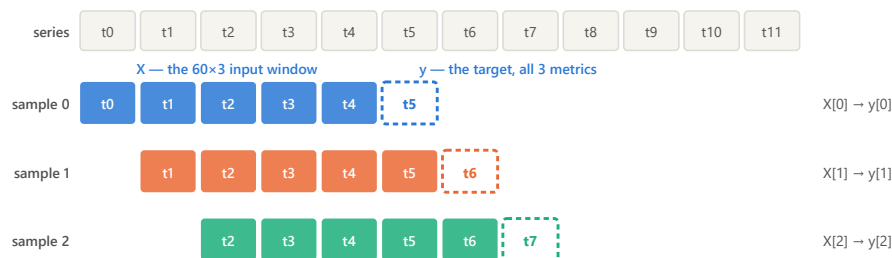
Proof it worked: training data scales to exactly 0.00–1.00, but test data reaches **1.91**. That can only happen if the scaler never saw the test data. The scaler is saved to `models/scaler.joblib` for the dashboard.

C. Make windows: 60 readings in, 1 reading out

Each example is 60 readings (5 hours) of CPU, memory and network; the answer is the next reading (5 minutes later). Windows are made inside each part separately, so no example mixes training and test data.

Sliding windows: 60 steps of history predict the 61st

Drawn with a window of 5 for legibility; the real window is 60 steps (5 hours) and the real target is 1 step (5 minutes) ahead.



The window slides one step at a time, so consecutive samples overlap by 59 of their 60 steps.

Two consequences: a split of R rows yields exactly $R - 60$ windows (the first 60 rows can only ever be history, never a target), and windowing must run separately per split — one window straddling the train/test boundary would leak the future into training.

D. Batches of 64

PART	ROWS	WINDOWS	BATCHES	SHUFFLED?
Training	6,048	5,988	94	Yes — only the order of windows
Validation	1,296	1,236	20	No
Test	1,296	1,236	20	No

Shuffling training windows helps learning. The readings inside each window stay in time order.

E. Checks

CHECK	EXPECTED	RESULT
Input batch shape	(64, 60, 3)	PASS

CHECK	EXPECTED	RESULT
Answer batch shape	(64, 3)	PASS
Un-scaled window 0 matches original CSV	difference ≈ 0	PASS 0.00024
Test data allowed outside 0–1	yes	PASS up to 1.91

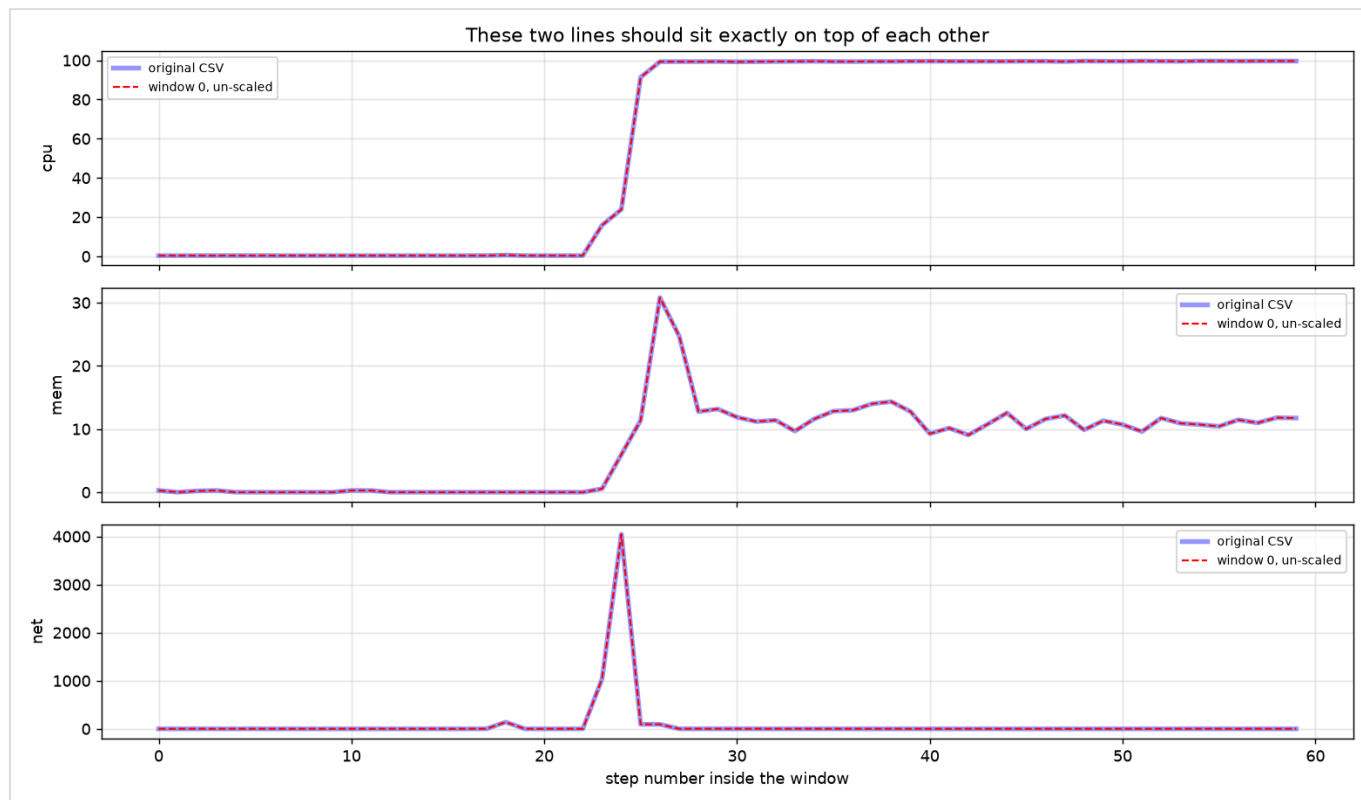


Figure 7. Original data and scaled-then-unscaled data sit exactly on top of each other.

6 Problems found and next steps

FIXED 1. THE SURVEY WAS COUNTING DAYS WHEN A VM WAS SWITCHED OFF

When a VM is switched off, the datacenter still writes a row every 5 minutes with every number at 0. The survey counted those as "0% busy", which dragged averages down. The first choice, **VM 214**, looked like mean 44.21% only because its last 4.6 days were switched off; while running it averaged **68.43%**, above our 60% limit. All six of the original top VMs had the same problem and none of them actually qualified.

Fix applied: `explore.py` now skips rows where the memory given to the VM is 0. Checked first on a 32-file sample: in every such row, all the other numbers are 0 too.

FIXED 2. THE SURVEY NEVER CHECKED THE TEST PERIOD

After fix 1 the new winner was **VM 200** — highest std of all, never switched off. But its last 4.5 days are flat zero, and that is exactly the slice kept back for testing. Training would have run fine and the final score would have meant nothing.

Fix applied: a fourth rule. The survey now measures the three parts the way preprocessing splits them and rejects any VM where one part does not move. That removed 20 more VMs, leaving 138 candidates, and **VM 270** is the top one.

STILL OPEN. THE TOP CANDIDATES ARE NEAR-COPIES

VMs 270, 249, 263, 247, 285 and 279 behave almost the same (probably one job spread over many machines), so choosing between them changes little. Ranking by std also favours on/off machines, because switching between 0% and 100% gives the largest possible std. VM 270 is still bimodal at the 5-minute level: 47% of readings at 0–2% and 36% above 98%. It does, however, have a clear daily rhythm (29% at 11:00, 62% at 20:00), which the earlier choice lacked.

Next steps

- 1 ~~Fix the survey and confirm the chosen VM.~~ Done — the chosen VM is now VM 270.
- 2 Add `README.md`, run `git init` and make the first commit.
- 3 Write `utils/generate_data.py` (synthetic data with known spikes).
- 4 **Step 1.4:** build and train the LSTM (input 60×3 , output 3).